

1.8: The $O(n \log n)$ algorithm to compute the convex hull of a set of n points in the plane that was described in this chapter is based on the paradigm of incremental construction: add the points one by one, and update the convex hull after each addition. In this exercise we shall develop an algorithm based on another paradigm, namely divide-and-conquer.

1.8 A: Let $P1$ and $P2$ be two disjoint convex polygons with n vertices in total. Give an $O(n)$ time algorithm that computes the convex hull of $P1 \cup P2$.
Circularly store vertices with $P1[0, \dots, m-1]$ CCW and $P2[0, \dots, k-1]$ CCW with $m + k = n$
 $\text{Orient}(a, b, c) = (b-a) \times (c-a)$

- Where > 0 is a left turn (counterclockwise), < 0 is a right turn (clockwise), and $= 0$ is collinear.

1. First need to get bridge between the polygons in $O(n)$

- Find the index (i) of the rightmost vertex of $P1$, using the max x and breaking ties by max y
- Find the index (j) of the leftmost vertex of $P2$, using the min x and breaking ties by min y

This scan will have a time of $O(m + k) = O(n)$

2. Find upper common tangent

Create a line through $(P1[i], P2[j])$ such that

- It supports $P1$ at $P1[i]$ and supports $P2$ at $P2[j]$ so that polygons are below the line at all points.

UpperTangent($P1, P2$)

- Set (i, j) from step 1 and repeat until there no change and upper common tangent is found.
- While the line $P1[i]P2[j]$ is not yet tangent to $P1$ at i , advance $i \leftarrow (i + 1) \% n$
- While the line $P1[i]P2[j]$ is not yet tangent to $P2$ at j , move $j \leftarrow (j-1) \% k$
- Each update improves the tangent height, until the upper tangent endpoints are found
- Upper tangent endpoints = (iu, ju)

Total moves of the endpoints being set = $O(m + k) = O(n)$

3. Find the lower common tangent

LowerTangent($P1, P2$)

- Reset (i, j) to rightmost of $P1$ and leftmost of $P2$
- Repeat until no change.
- While not tangent to $P1$ at i for lower support, move $i \leftarrow (i - 1) \bmod m$
- While not tangent to $P2$ at j for lower support, move $j \leftarrow (j + 1) \bmod k$

Total moves = $O(m + k) = O(n)$ and the lower tangent endpoints are (il, jl)

4. Create the hull by walking outside

Boundary of $CH(P1 \cup P2)$ includes:

- Upper tangent edge $P1[iu] \rightarrow P2[ju]$, outer chain of $P2$ from ju to jl in CCW order, lower tangent edge $P2[jl] \rightarrow P1[il]$, and the outer chain of $P1$ from il to iu in CCW order

Output the vertices in counterclockwise order by starting at $P1[iu]$, adding $P2[ju]$, and walking the entirety of $P2$ before adding $P1[jl]$ and walking the entirety of $P1$.

This will then touch each vertex at most once with duplicate handling and will result in $O(n)$ algorithm time.

1.8 B: Use the algorithm from part a to develop an $O(n \log n)$ time divide-and conquer algorithm to compute the convex hull of a set of n points in the plane

1. Sort all of the points in the array by increasing x (y tiebreaker)
2. Define function $Hull(p[l \dots r])$ that returns convex hull of points as a polygon listed counterclockwise

Base Case: If $r - l + 1 \leq 3$, compute the hull directly

- 1 point $\rightarrow$ that point, 2 points $\rightarrow$ segment endpoints, 3 points $\rightarrow$ triangle
- Cost = $O(1)$

Recursive Case:

- Split by median index, $PL = P[l \dots m]$ and $PR = P[m + 1 \dots r]$
- Compute $HL = CH(PL)$ and $HR = CH(PR)$
- Return $MergeHulls(HL, HR)$ using part A merge algorithm in the $O(n)$ time

3. Part A merge algorithm

- Find upper common tangent between HL and HR , find lower common tangent, and output the merged hull through walking the outer chain of HL and the outer chain of HR between the tangent endpoints.
- Time = $O(|HL| + |HR|)$ and hull sizes are less than point counts, so $O(n)$ per merge level.

Total Time:

- Each call does 2 recursive calls on $n/2$ and 1 merge in $O(n)$ so $T(n) = 2T(n/2) + O(n)$
- Master theorem: $T(n) = O(n \log n)$

2.12: Let S be a set of n triangles in the plane. The boundaries of the triangles are disjoint, but it is possible that a triangle lies completely inside another triangle. Let P be a set of n points in the plane. Give an $O(n \log n)$ algorithm that reports each point in P lying outside all triangles.

1. Create planar subdivision DCEL of all triangle edges in $O(n \log n)$

- For each triangle T_i with vertices (a_i, b_i, c_i) , add the three edges in format:
- (a_i, b_i) , (b_i, c_i) , (c_i, a_i)
- Subdivision complexity is $O(n)$

- Create a DCEL storing including half-edges with Next, Twin, Prev, and IncidentFace fields
 - Use plane sweep paradigm with event queue and status tree and update at event points.
 - Process the segment endpoints as events $\rightarrow$ total time is $O(n \log n)$ from sorting and $O(\log n)$ for each insert and delete in status tree
 - Now have DCEL that represents all faces
2. Create point location structure over subdivision in $O(n \log n)$
- Sort all unique vertex x values
 - During the sweep, record the sweep-line status order for each vertical slice.
 - For each slice, vertical order of intersecting edges partitions slice into interval regions
 - Then link each region to face record in the DCEL
 - Store this in a binary search tree with the slices sorted by x value. The second level of the BST would be the ordering if the x values match, then sorted by y value

Time remains $O(\log n)$ since there are n number of slices = $O(n)$ and updates happen at $O(n)$ events and each update costs $O(\log n)$

3. Identify which face is outside of the triangles
- Unbounded face in DCEL with outer boundary at infinity.
 - Store this with a label like OUTSIDE_FACE

4. Query all points and report the ones outside

For each of the query points $p \in P$

- Use point location to find containing face $f(p)$ in $O(\log n)$
- If $f(p) = \text{OUTSIDE_FACE}$, then report p
- If p lies on an edge, treat it as inside
- Continue until the point p is found to be outside.

Total query time: $n * O(\log n) = O(n \log n)$

Total time for all steps:

Building subdivisions with sweep and DCEL $\rightarrow O(n \log n)$

Building point-location search structure $\rightarrow O(n \log n)$

Query n points $\rightarrow O(n \log n)$

Overall time: $O(n \log n)$

2.14: Let S be a set of n disjoint line segments in the plane, and let p be a point not on any of the line segments of S. We wish to determine all line segments of S that p can see, that is, all line segments of S that contain some point q so that the open segment pq doesn't intersect any p line segment of S. Give an $O(n \log n)$ time algorithm for this problem that uses a rotating half-line with its endpoint at p.

1. Convert each segment into angular interview

For each segment $s = ab$ —

- Compute polar angles around p:
- $\alpha = \angle(pa)$ and $\beta = \angle(pb)$
- Ray from p intersects s for a contiguous interval of angles between α and β
- If it wraps around, split it into two intervals $[\alpha, 2\pi) \cup [0, \beta]$
- Create events at the interval endpoints (insert s and delete s at the end angle)

This results in $O(n)$ intervals and $O(n)$ events with at most $2n$ endpoints

2. Sort all of the event angles

Sort all of the event angles with tie handling being consistent.

This results in $O(n \log n)$ time

3. Maintain and update the rotating-ray status structure

Maintain the set T of segments that are currently intersected by the ray, which uses balanced BST that can have inserts and deletes in $O(\log n)$

- For θ angle, each active segment $s \in T$ is intersected by ray at some point $xs(\theta)$
- Its ray-distance key is $k(s, \theta) = \text{distance}(p, xs(\theta))$
- Store the segments in T ordered by this key, then minimum key segment is visible in angular interval
- Splay trees give amortized $O(\log n)$ per operation, so balanced-tree justified.

4. Sweep/rotate and report the visible segments

Initializer $T = \emptyset$ and empty output set V and process the events in increasing angle order. For each event angle θ_i

- Apply updates to T :
- For every segment whose interval starts at θ_i , insert into T
- For every segment whose interval ends at θ_i , delete from T
- Query the front segment
- If $T \neq \emptyset$, let s be the segment with minimum $k(s, \theta)$, the nearest intersection along ray
- Add s to V as it is visible for all angles between θ_i and the next event θ_{i+1}
- At the end, output all visible segments in V

Total time = $O(n \log n)$ with space $O(N)$ for events and the binary search tree.

Building intervals and events = $O(n)$

Sort events: $O(n \log n)$

Insert and delete segments: $O(1)$ times is $O(n)$ updates, each $O(\log n)$ in balanced BST